

Topic : Compiler Construction

Chapter 1 : Introduction to Compiler

- 1.1: What is Compiler?**
- 1.2: History & Evolution of compilers**
- 1.3: Language Processors**
- 1.4 : Compiler vs Interpreter**
- 1.5: Structure of a Compiler**
- 1.6 : Phases of Compiler**
- 1.7: Front End & Back End of Compiler**
- 1.8: Passes of Compiler (Single pass, Multi pass)**
- 1.9 : Bootstrapping of Compiler**
- 1.10: Challenges in Compiler Design**
- 1.11: Modern compiler Examples**
- 1.12: Application of compiler Technology**
- 1.13: Error Handling in Compiler**

Chapter 2 : Lexical Analysis

- 2.1: Role of Lexical Analyzer**
- 2.2 : Tokens, Lexemes, Patterns**
- 2.3 : Specification of Tokens**
- 2.4 : Regular Expressions**

- 2.5 : Regular Definitions**
- 2.6 : Regular Languages**
- 2.7 : Finite Automata**
- 2.8 : Transition Diagrams**
- 2.9 : Conversion of NFA to DFA**
- 2.10 : Minimization of DFA**
- 2.11 : Design of Lexical Analyzer**
- 2.12 : input Buffering**
- 2.13 : Symbol Table Interaction**
- 2.14: Longest Match & Rule priority**
- 2.15:Error handling in Lexical Analyzer**
- 2.16: Efficiency Issues in Lexical Analyzer**
- 2.17: Lex Tool / Lexical Analyzer Generator (LEX / Flex)**
- 2.18: Modern Lexer Generators**
- 2.19: Unicode & UTF-8 Handling**
- 2.20 : Lexical Analysis in Modern Compilers**

Chapter 3 : Syntax Analysis

- 3.1:Introduction & Role of Syntax Analyzer**
- 3.2: Context Free Grammar (CFG)**
- 3.3: Derivations (Leftmost & Rightmost)**
- 3.4: Parse Trees**
- 3.5: Ambiguity in Grammar**
- 3.6: FIRST and FOLLOW Sets**

- 3.7: Left Recursion**
- 3.8: Left Factoring**
- 3.9: Top-Down Parsing**
- 3.10: Recursive Descent Parsing**
- 3.11 : Predictive Parsing**
- 3.12: LL(1) Parsing Table Construction**
- 3.13: Bottom-Up Parsing**
- 3.14: Shift-Reduce Parsing**
- 3.15: LR parsing**
- 3.16: Operator Precedence Parsing**
- 3.17: Parser Generators**
- 3.18: Error handling in syntax Analysis**

Chapter 4 : Semantic Analysis

- 4.1: Role of Semantic Analyzer**
- 4.2: Symbol table**
- 4.3: Scope & Binding**
- 4.4: Name Resolution**
- 4.5: Declaration & Consistency Checking**
- 4.6: Type Systems**
- 4.7: Type Expression**
- 4.8: Type Checking**
- 4.9: Type Conversion**
- 4.10: Attributes & Attributes Evaluation**
- 4.11: Syntax Directed Definition (SSD)**

- 4.12: Attribute Grammars**
- 4.13: Syntax Directed Translations (SDT)**
- 4.14: Semantic Error Handling**
- 4.15: Intermediate Representation (Introduction)**
- 4.16: Abstract Syntax Tree (AST)**

Chapter 5 : Intermediate Code Generation

- 5.1: Need for Intermediate Code**
- 5.2: Role of IR in Compiler**
- 5.3: Types of Intermediate Representation**
- 5.4: Three Address Code (TAC)**
- 5.5: Representation of TAC**
- 5.6: Syntax-Directed Translation for ICG**
- 5.7: Translation of Expressions**
- 5.8: Translation of Boolean Expressions**
- 5.9: Short-Circuit Evaluation**
- 5.10: Translation of Statements**
- 5.11: Control Flow Statements**
- 5.12: Backpatching**
- 5.13: Abstract Syntax Tree to IR Conversion**
- 5.14: DAG Representation for Expression**

Chapter 6 : Code Optimization

- 6.1: Introduction to Code Optimization**
- 6.2: Need for Optimization**

- 6.3: Goals of Optimization**
- 6.4: Types of Optimization**
- 6.5: optimization Scope**
- 6.6: Basic Blocks**
- 6.7: Control flow Graph (CFG)**
- 6.8: DAG Representation for Expression**
- 6.9: Peephole Optimization**
- 6.10: Data flow Analysis**
- 6.11: Common Subexpression Elimination**
- 6.12: Constant Folding**
- 6.13: Constant Propagation**
- 6.14: Constant Propagation**
- 6.15: Dead Code Elimination**
- 6.16: Strength Reduction**
- 6.17: Loop Optimization**
- 6.18: Global optimization**

Chapter 7 : Code Generation

- 7.1: Introduction to Code Generation**
- 7.2: Issues in Code Generation**
- 7.3: Target Machine Architecture**
- 7.4: Addressing Mode**
- 7.5: Instruction Selection**
- 7.6: Register Allocation & Assignment**
- 7.7 : Evaluation order**

- 7.8: Code Generation for Expressions**
- 7.9: Code Generation for Boolean Expression**
- 7.10: Code Generation for Control statement**
- 7.11: Code Generation for Procedure Calls**
- 7.12: Calling Conventions**
- 7.13: Stack frame & Activation Record Layout**
- 7.14: Runtime Storage Management**
- 7.15: Instruction Scheduling**
- 7.16: Spill code handling**
- 7.17: Error handling and debugging support**

Chapter 8 : Runtime Environment

- 8.1: Introduction to Runtime Environment**
- 8.2 :Source Language Issues**
- 8.3: Binding Time**
- 8.4:Storage Organization**
- 8.5 :Storage Allocation Strategies**
- 8.6: Procedure Call and Return Mechanism**
- 8.7 :Scope Rules & Accessing Non-local Variables**
- 8.8: Storage Allocation Strategies**
- 8.9: Dynamic Storage Management**
- 8.10 :Runtime Support for Control Flow**
- 8.11: Runtime Environment for Object-Oriented languages**
- 8.12 :Error Handling at Runtime**
- 8.13 : Garbage Collection Techniques**

Chapter 9 : Error Handling

9.1: Introduction to Error Handling

9.2: Types of Errors

9.3: Error Detection and Reporting

9.4: Error Recovery Techniques

9.5: Error Handling in Different Compiler phases

9.6: Error Handling Strategies

9.7: Error Handling vs Exception Handling

9.8: Design Issues in Error Handling

Chapter 10: Modern Compiler Architecture and Industry practices

10.1: Modern Compiler Architecture

**10.2 : Multi-Level Intermediate Representation
(Multi-IR Design)**

10.3: Profile-Guided Optimization (PGO)

10.4: Link-Time Optimization (LTO)

10.5: Incremental & Interactive Compilation

10.6: Just-In-Time (JIT) & Hybrid Compilation

10.7: Heterogeneous & Parallel Compilation

10.8: Build Systems & Compiler Toolchains

10.9: Compiler Testing & Validation

10.10: Security in Compiler Design

10.11: Static & Dynamic Compilation

10.12: Cross Compilation & Embedded Systems

10.13: LLVM & Modern compiler frameworks

10.14: Cloud & Distributed Compilation

10.15: AI & ML Compiler Concepts

